

Rapport final — Simulateur SHDL (ARCHIDU7)

Projet Long SHDL

Contenu rédigé par Arthur Madar — Diagrammes de classes réalisés par Alexis Briend

23 mai 2026

Rappel du sujet

Créer une application capable de simuler des circuits logiques à partir de scripts SHDL.

Features

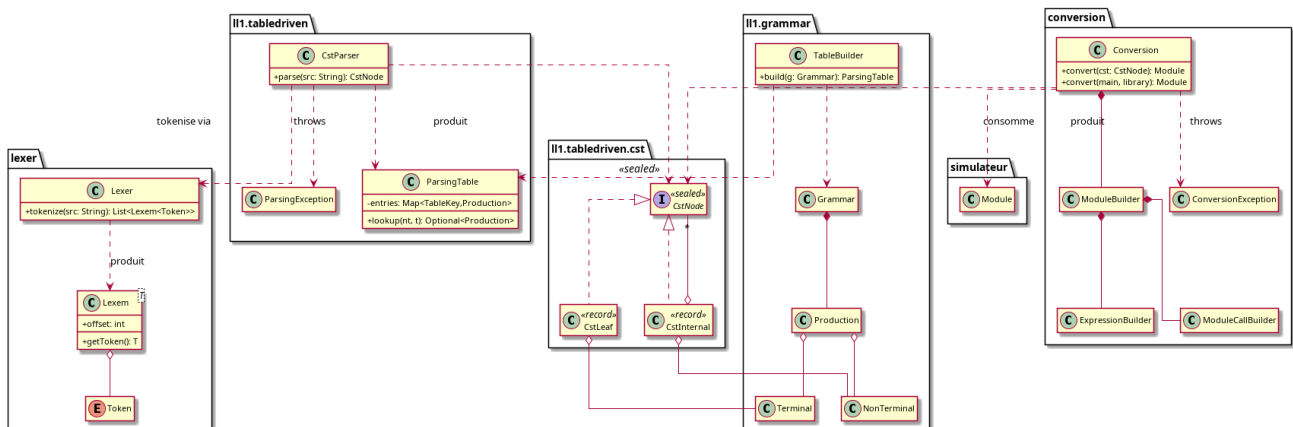
- **Interprétation du langage et modélisation** : pouvoir transformer un script SHDL en une modélisation de circuit logique. Cette feature est complétée, elle avait été principalement faite au sprint 1 et 2 mais il restait des choses à ajouter pour le sprint 3 (appel de sous module principalement).
- **Simulation** : pouvoir simuler un circuit logique avec une interface graphique dédiée. Cette feature est complétée, elle avait été principalement faite au sprint 1 et 2 mais il restait des choses à ajouter pour le sprint 3.
- **Éditeur de Texte** : un éditeur de texte adapté au langage SHDL (coloration syntaxique/autocomplétion). Il lui manque une partie de l'autocomplétion que nous n'avons pas pu livrer à temps. Cette feature à été implémenté majoritairement durant le sprint 2 mais quelques ajouts et corrections ont pu être effectués au sprint 3.
- **Interface graphique** : l'interface graphique de l'application. Celle-ci à été principalement faite au sprint 3, les sprints précédents ne fournissait qu'un squelette.
- **Script de test** : pouvoir lancer des script de test simple sur une simulation. Seul l'essentiel a pu être terminé, d'autres fonctionnalités prévues n'ont pas pu être implémentées. Cette features à été démarré durant le sprint 3.

Paquetages

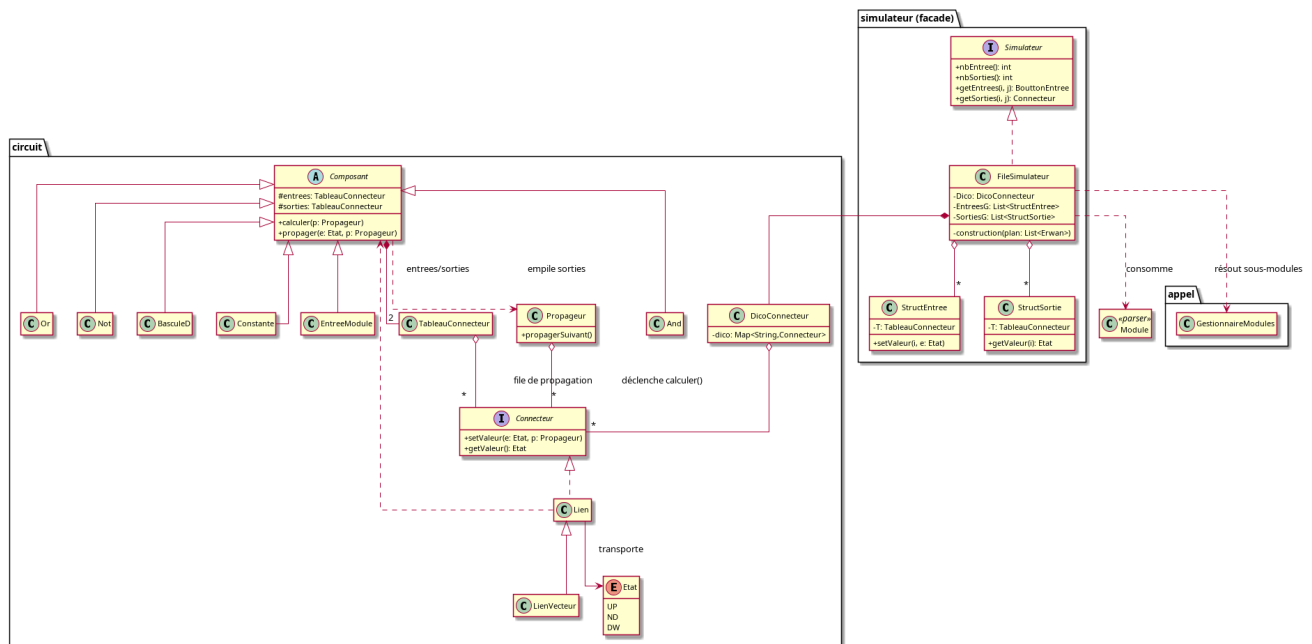
- | | | |
|--------------------------|--------------------------|--------------------------------|
| • editeur | • parser.automate | • parser.ll1.tabledriven.cst |
| • editeur.autocomplétion | • parser.conversion | • parser.ll1.tabledriven.table |
| • editeur.colorisation | • parser.lexer | • util |
| • boutons | • parser.regex | • simulateur |
| • erwan | • parser.ll1 | • simulateur.affichage |
| • sauvegarde | • parser.ll1.grammar | • simulateur.appel |
| • parser | • parser.ll1.tabledriven | • simulateur.scriptTest |

Diagramme de classe

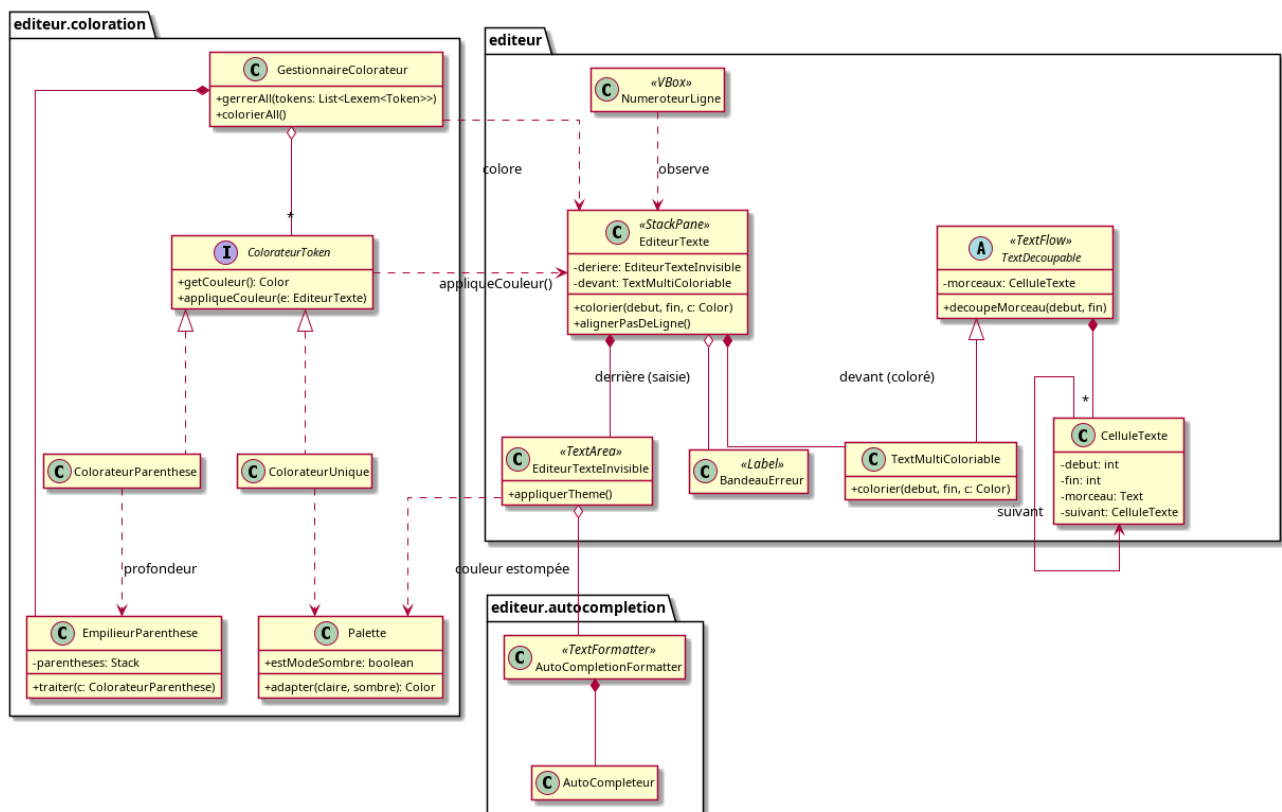
Le parser



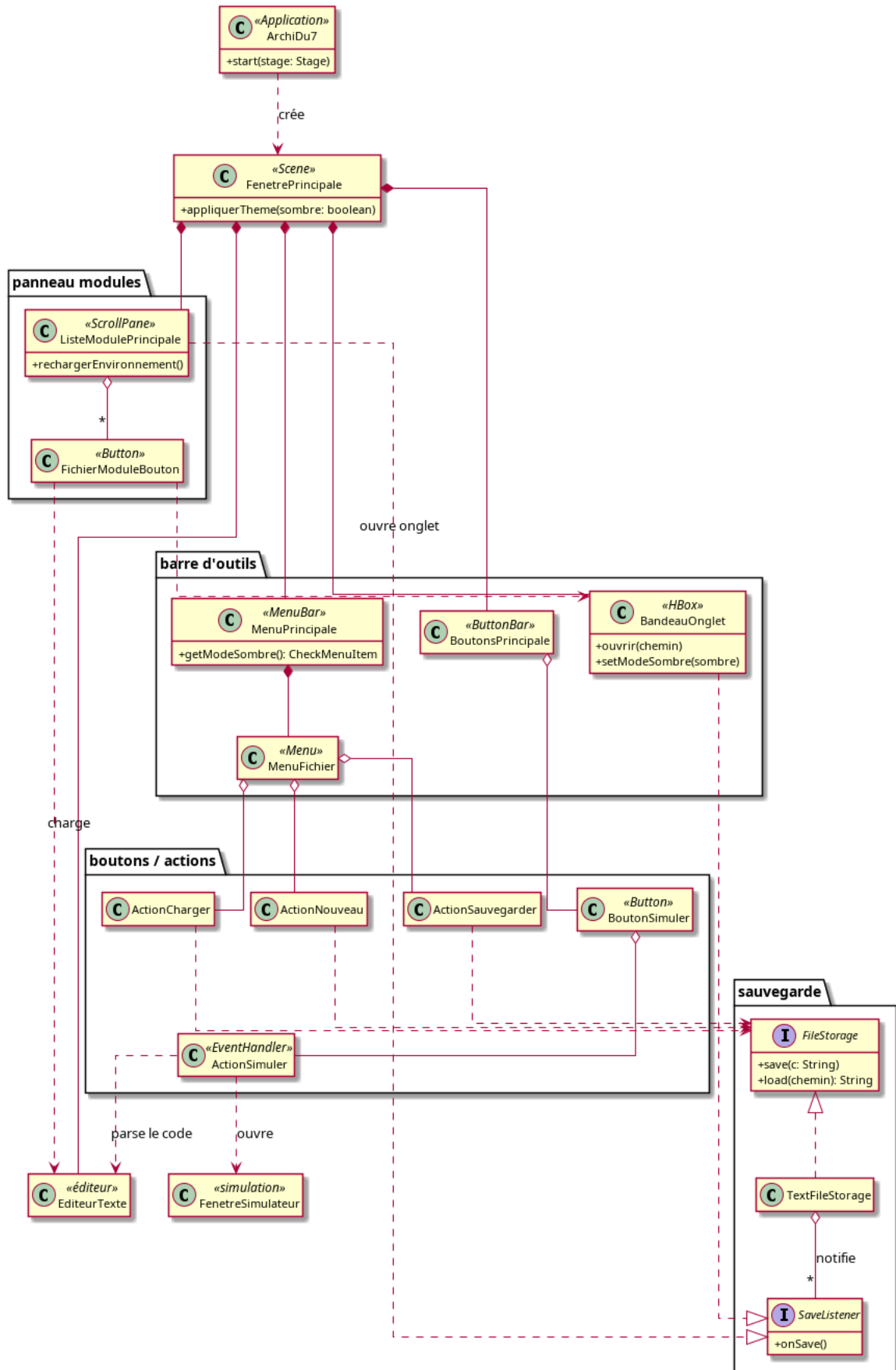
La simulation



L'éditeur de texte



Architecture globale de l'interface



Dynamique du système

Choix de conception

- Utilisation de javafx afin de pouvoir utiliser du css dans le code.
- Les simulations apparaissent dans une fenêtre séparée du reste.
- Les fichiers SHDL sont scannés d'un même dossier `modules/`.
- La transformation du module en circuit n'est calculée que lorsque l'on veut lancer sa simulation.

Problèmes de conception rencontrés

- Initialement, l'état des potentiel dans le circuit était calculé en mettant à jour tous les potentiels mais cela créait un problème de logique séquentielle donc nous avons mis en place une méthode de calcul par propagation (parcours en largeur).
- La classe `TextArea` en javafx ne permet de colorier des morceaux indépendant de texte. Nous avons donc décidé d'afficher du texte coloriable par morceau (de notre propre implémentation) par dessus le `TextArea` pour donner l'illusion d'une coloration syntaxique.

Organisation de l'équipe

- Nous avons découpé l'effectif entre la partie interface graphique/éditeur de texte et la partie modélisation/simulation.
- Nous nous répartissions les tâches à l'aide de Jira.
- À partir du sprint 2, nous faisons un appel vocal tous les mercredis pour faire un point sur l'avancement du projet.
- Nous avons fait un groupe de discussion discord pour nous partager les avancées de chaque parties et nous poser des questions.